

Simulazione del comportamento di stormi

Redatta da: Spallacci Leonardo, Telespan Matteo Mihnea

Introduzione:

In questo progetto di simulazione del comportamento di stormi, si vuole implementare tramite codice C++ il comportamento degli stormi di uccelli in volo in uno spazio bidimensionale. Questo programma si basa sul modello realizzato da Craig Reynolds nel 1986, in cui gli uccelli, che chiameremo boids, si muovono e modificano il loro volo a seconda di tre regole: separazione, che allontana il boid da quelli troppo vicini; coesione, che fa avvicinare il boid verso il baricentro di quelli limitrofi; allineamento, che rende la traiettoria di volo di un boid simile a quella dei vicini. Per la realizzazione di questo progetto, dovendo lavorare a distanza, abbiamo scelto di sfruttare la piattaforma di programmazione collaborativa GitHub. Ci siamo dunque creati un account sul sito online, e una volta creata la repository comune, abbiamo diviso il progetto in più rami in modo che ognuno potesse lavorare separatamente sul codice ma che fosse comunque sempre visibile anche all'altro. Questo ci ha consentito di separare i compiti e di controllare volta per volta le varie modifiche che sono state fatte. La nostra repository è pubblica ed è all'indirizzo: <https://github.com/leospalla/progetto>. Da essa è possibile anche visualizzare la cronologia dei nostri commit sui rispettivi branche per vedere l'evoluzione del progetto.

Codice e scelte d'implementazione:

Il programma è costituito da tre header file: `vector.hpp`, `boid.hpp`, `flock.hpp` e dai relativi file sorgente: `vector.cpp`, `boid.cpp`, `flock.cpp`. Negli header file vi sono le definizioni delle classi e la dichiarazione delle funzioni, nei file sorgente vi è la definizione delle funzioni. Per ogni header file e codice sorgente vi è il corrispondente file di testing: `vector.test.cpp`, `boid.test.cpp` e `flock.test.cpp`. Infine, vi è un ulteriore file che richiama la funzionalità: `main.cpp`. La suddivisione in tre header file corrisponde alla suddivisione delle varie funzioni del progetto in tre classi con i corrispondenti namespace. La suddivisione in classi segue la seguente logica: la classe "Vector" contiene gli strumenti matematici necessari per sviluppare il progetto, la classe "Boid" descrive il singolo uccello con tutte le caratteristiche e proprietà che deve avere ed infine la classe "Flock" crea lo stormo e ne calcola alcune caratteristiche.

Classe Vector

La classe Vector serve per costruire un vettore bidimensionale di numeri razionali. I membri privati sono dunque due double che corrispondono alle due coordinate. Entrambi sono inizializzati a 0. Nella parte public della classe sono presenti innanzitutto i costruttori, cioè delle funzioni speciali presenti nelle classi che quando chiamate inizializzano un oggetto di quel tipo (infatti il loro nome è lo stesso di quello della classe), impostando lo stato iniziale delle sue variabili membro. Nella nostra classe Vector i costruttori sono tre: il costruttore di default che inizializza un vettore con coordinate nell'origine, il costruttore che inizializza il vettore con le sue coordinate nello spazio, ed infine il costruttore copia, il quale è stato definito esplicitamente per evitare che il compilatore commetta errori generandolo in automatico quando viene usato in altre funzioni (`getPosition` e `getVelocity` della classe Boid che vedremo dopo). Vi sono poi le implementazioni dei vari operatori che rendono possibili le operazioni fra oggetti di tipo Vector. Questi sono stati implementati di modo che prendessero delle referenze costanti di un altro Vector (oppure un double nel caso di moltiplicazione e divisione per scalare) perché operazioni come somma o differenza non devono modificare gli oggetti su quali sono state fatte le operazioni (ad eccezione degli operatori `=` e `+=` che invece li modificano). Abbiamo poi le funzioni membro della classe Vector, utili per lo sviluppo del progetto. Naturalmente le funzioni come `normalize`, `set` e `limit`, dovendo esse modificare lo stato interno del Vector, non sono state definite come `const`. Le funzioni che non modificano i membri privati sono state dichiarate come `const`. Un

esempio è la funzione `distance` che calcola la distanza tra il `Vector` su cui è chiamata e il `Vector` preso come parametro. Per implementare queste funzioni è stata usata la libreria `"cmath"` inclusa nell'header file.

Nota: da qui in poi useremo `"Vector"` per parlare dei vettori bidimensionali definiti nella classe `Vector`, mentre `"vector"` per indicare i container proposti dalla standard library.

Classe Boid

La classe `Boid` costruisce l'oggetto che corrisponde al singolo uccello. Come membri privati vi sono due `Vector` che corrispondono alla posizione e alla velocità del Boid. Si ha poi un `double` che rappresenta il modulo massimo della velocità permesso ai Boid. E' necessario fissare un valore massimo alla velocità per evitare che, tramite la somma delle velocità date dalle tre regole di volo, i Boid assumano velocità troppo elevate e diventino fuori controllo. Quest'ultima infatti è un membro privato dichiarato come costante in quanto abbiamo ritenuto non fosse necessario modificarla. Entrando nel public abbiamo come `double` i parametri delle regole, già inizializzati per poter eseguire direttamente i test. Il `perceptionRadius` è la distanza per cui si attivano le regole di coesione e allineamento, `separationDistance` è invece la distanza alla quale si attiva la regola di separazione. Gli altri sono i parametri veri e propri delle regole, ovvero dei fattori di proporzionalità che permettono di stabilire quanta influenza si vuole dare ad ogni regola. Per ognuno di questi parametri vi è una funzione `set`, così come vi è una funzione `set` anche per la posizione e la velocità. Passiamo poi alle definizioni dei costruttori. Si ha il costruttore di default che inizializza il Boid con una posizione e una velocità casuale, in cui il range della posizione è stabilito dal programma (anche in questo caso si è scelto un intervallo arbitrario per poter eseguire con più facilità i test). Il secondo costruttore è quello che inizializza il Boid con una posizione e una velocità casuale, in cui però il range della posizione è stabilito dall'utente. Si hanno poi a seguire altri costruttori utilizzati nei file di test. Per l'inizializzazione delle variabili casuali vi sono delle funzioni libere apposite che generano numeri casuali nell'intervallo voluto. Per questo scopo abbiamo incluso la libreria `"random"`. Nei costruttori in cui la velocità viene generata casualmente è presente un ciclo `while` nel quale semplicemente si evita la remota possibilità che un Boid abbia velocità nulla, in quanto abbiamo pensato fosse irrealistico (è stato aggiunto un'ulteriore controllo che genera un messaggio di errore qualora dopo 10 tentativi la velocità generata a random sia ancora nulla attraverso la libreria `"stdexcept"`). Sono presenti poi gli overload degli operatori di assegnazione e di uguaglianza fra Boid necessari per le funzioni implementate nella classe `flock`. Proseguendo si hanno tutta una serie di funzioni utili come `getPosition`, `getVelocity` e `getMaxSpeed` che permettono di accedere ai membri privati, mentre `getSpeed` restituisce la norma della velocità del Boid. La funzione `centerOfMass` calcola il centro di massa dei Boid vicini al Boid su cui viene chiamata. Quest'ultima è necessaria per la regola di coesione e inizialmente non era separata dalla funzione `cohere` ma era fusa con essa. Successivamente però abbiamo voluto separarla perché è buona regola dividere le funzioni indipendenti e rendere il codice sorgente più leggibile. La struttura di `centerOfMass` segue la stessa logica delle funzioni per le regole di volo che è la seguente: prende come parametro un `vector` di Boid della standard library, motivo per cui si è inclusa la libreria `"vector"`, e con un ciclo `for-each` che itera su tutti gli elementi del `vector` controlla la distanza fra il Boid chiamante e tutti gli altri. Se questa distanza è maggiore di 0 (per evitare che il Boid conti sé stesso nel calcolo), e minore di una certa distanza di attivazione, il Boid viene considerato come vicino. A questo punto nella funzione `centerOfMass` si fa la sommatoria delle posizioni dei Boid vicini, la quale viene poi divisa per il numero di vicini. Nella funzione `cohere` semplicemente una volta chiamata la funzione `centerOfMass` si sottrae alla posizione del centro di massa ottenuta la posizione del Boid chiamante, e si moltiplica il `Vector` risultante per il fattore di coesione. La stessa logica viene utilizzata nella funzione `align` dove però si ha a che fare con le velocità dei Boid vicini. Per la funzione `separate` al posto del `perceptionRadius` come distanza di attivazione viene utilizzata la `separationDistance`, perché essa ha il compito di evitare che i Boid si sovrappongano. `Separate` ha comunque una logica simile alle altre due regole di volo, solo che essa restituisce un vettore con direzione opposta rispetto alla posizione dei vicini. Come ultima funzione abbiamo `borders`, la quale prende come parametro la lunghezza del lato del quadrato che costituisce lo spazio in cui i Boid si possono muovere e fa sì che i Boid rimangano sempre

all'interno di tale spazio, il quale è centrato nell'origine (per questo la lunghezza viene divisa per 2). In particolare fa in modo che quando un Boid esce dallo spazio esso si "teletrasporta" immediatamente dalla parte opposta, in modo da ottenere uno spazio di simulazione toroidale. Va specificato che una delle ultime cose che abbiamo provato a fare è stato quello di ottimizzare l'implementazione delle regole di volo attraverso l'utilizzo di oggetti funzione (le lambda functions) e algoritmi della standard library come accumulate per velocizzare i calcoli delle distanze e le sommatorie. Una volta terminato però, siccome avevamo reso il codice a nostro avviso eccessivamente complesso e più difficile da leggere senza aver guadagnato granché in termini di prestazione, abbiamo ritenuto fosse meglio ritornare alle implementazioni precedenti.

Classe Flock

La classe Flock crea il tipo che corrisponde allo stormo di uccelli quindi di Boid. Come membri privati della classe abbiamo un vector di Boid che rappresenta lo stormo e la minima unità di variazione di tempo che il programma utilizza per l'evoluzione della simulazione. Non abbiamo ritenuto necessario rendere possibile la sua modifica all'utente ed è inizializzata a 0,1s. Nel public abbiamo innanzitutto il costruttore che per Flock è unico e semplicemente inizializza un vector di Boid vuoto. Si hanno poi i soliti metodi per accedere ai metodi privati: getBoids e getDeltaTime. Procedendo si hanno i metodi per l'aggiunta e la rimozione di Boid allo stormo. La funzione addBoid sfrutta la funzione push_back della libreria vector mentre la funzione removeBoid usa degli iteratori per trovare il Boid da rimuovere nello stormo e rimuoverlo tramite la funzione erase. Le funzioni updateVelocity e updatePositions aggiornano rispettivamente le velocità e posizioni nel modo appropriato, sulla base delle regole di volo. Per farlo si itera sull'indice di ogni Boid, si aggiornano le velocità o le posizioni di ogni Boid, successivamente si aggiungono questi valori ad un nuovo vector e solo dopo che tutti i Boid sono stati aggiornati si settano velocità o posizione nel vector m_boids. Inizialmente il ciclo settava subito i valori aggiornati in m_boids ma in questo modo una volta aggiornato il primo Boid esso influiva sui prossimi aggiornamenti e così gli altri. Per questo abbiamo deciso di cambiare implementazione e fare in modo che gli aggiornamenti dei Boid avvenissero nello stato in cui sono quando viene chiamata la funzione, così che l'aggiornamento di alcuni non influenza quello di altri. Si passa ora alle funzioni statistiche. La funzione averageDistance calcola la distanza media su tutti i Boid. Lo fa con un doppio ciclo for che itera gli indici di modo da trovare tutte le coppie di Boid, calcolare la loro distanza e farne poi la media. All'inizio questa funzione era stata implementata in un modo svantaggioso in quanto calcolava la distanza tra ogni singolo Boid, ma all'interno dei cicli for questi calcoli venivano ripetuti anche per Boid la cui distanza era già stata calcolata. A progetto terminato abbiamo poi tentato di modificare il codice correggendo i cicli for e abbiamo verificato grazie ai comandi che mostravano il tempo di esecuzione del programma (time ./build/main) che la nuova implementazione era senza dubbio migliore perché più rapida. La funzione standardDeviationDistance seguendo la stessa logica di averageDistance calcola la deviazione standard della distanza tra tutti i boids. La funzione averageSpeed calcola invece la media dei moduli delle velocità. Non essendoci la necessità di individuare le coppie di Boid non serve il doppio ciclo for. La funzione standardDeviationSpeed calcola la deviazione standard dei moduli delle velocità con la formula classica. Va specificato che queste funzioni (specialmente quella delle distanze) si apprezzano di più nei casi in cui si ha a che fare con uno stormo solo. La funzione countBoidsInFlock invece è più espressiva quando si ha a che fare con più stormi in un'unica simulazione. L'obiettivo è quello di raggruppare i Boid che si trovano all'interno della distanza di percezione specificata in stormi separati e quindi restituire un vettore contenente il numero di Boid in ciascuno di questi stormi. La funzione itera su tutti i Boid presenti e sfruttando un vector di bool e la libreria "stack" individua i Boid vicini. Il vector di bool serve per tenere traccia dei Boid che sono già stati visitati durante il processo di formazione degli stormi. La pila di Boid inizializzata con std::stack invece viene utilizzata con un ciclo while: finché essa non è vuota si estrae l'indice del Boid corrente dalla cima della pila e se non è stato visitato si incrementa il numero di Boid nel flock corrente, poi lo si contrassegna come visitato e si cercano altri Boid che siano nello stesso stormo. Si fa cioè in modo che uno stormo grande non conti come stormi multipli, anche se i Boid agli estremi dello stesso flock risultano distanti. Inizialmente la funzione era stata implementata senza l'uso della libreria "stack" ma si andava così a creare un problema di ordine: se i

Boid dello stesso stormo erano disposti in modo che non si alternassero con Boid di altri stormi, la funzione funzionava correttamente; ma se così non era assegnava alcuni Boid allo stormo errato. Non siamo riusciti a risolvere questo problema usando solo gli strumenti della libreria "vector" perché poi si andavano a creare errori di comportamento indefinito e di "buffer-overflow" e quindi abbiamo sfruttato la libreria "stack". Infine, abbiamo la funzione `simulate` che semplicemente viene chiamata dal main e per ogni passo della simulazione in base a quanto si vuole far andare avanti mette a schermo i dati collettivi sullo stormo quindi l'andamento della distanza media e dei moduli delle velocità e delle loro deviazioni standard così come il numero di stormi con il corrispondente numero di Boid.

Strategia di Test:

Per controllare che le funzioni implementate facessero effettivamente quello che era richiesto e che si comportassero bene anche in casi specifici, si è scelto di effettuare svariati test. Lo strumento di testing che si è scelto è Doctest. Esso è una libreria di testing per il linguaggio C++ progettata per semplificare il processo di esecuzione di unit tests, infatti grazie a Doctest è possibile testare piccole porzioni di codice verificando i comportamenti attesi in svariate situazioni. In questo progetto si è scelto di scrivere un file di testing per ognuno dei file sorgente creati, per testare dunque le tre classi Vector, Boid e Flock. Per testare la classe Vector, si è verificato innanzitutto che l'overloading degli operatori fosse corretto, e dopodiché si sono anche testate tutte le funzioni utilizzate nel resto del progetto (come `distance`, `set`, `limit` e altre). Non essendo funzioni eccessivamente complesse non è sembrato necessario svolgere troppi test che sarebbero potuti apparire ridondanti. Nel testare invece la classe Boid sono stati necessari più test per accertarsi che le regole di volo implementate descrivessero correttamente il comportamento dei Boid desiderato. Innanzitutto si sono testati i vari costruttori e ci si è assicurati che le funzioni di generazione casuale di numeri fossero corrette e che l'invariante di classe della velocità massima venisse in ogni caso rispettato. Dopodiché siamo passati al testare le tre regole di volo: separazione, coesione (con annessa la funzione `centerOfMass`) e allineamento. In origine si era provato a testare tutte e tre le funzioni contemporaneamente includendole in una funzione `update`, ma poi per semplicità e per evitare errori abbiamo deciso di testarle singolarmente in tutti i casi possibili, in particolare concentrandosi sulle distanze reciproche tra i vari Boid. Abbiamo quindi controllato che qualora i Boid fossero troppo lontani le regole di volo non si sarebbero dovute attivare, mentre se considerati vicini le funzioni avrebbero restituito i valori calcolati mediante le formule delle regole di volo. Ovviamente i valori attesi sono stati calcolati a partire da parametri predefiniti, in quanto essi non vanno a modificare il funzionamento interno della funzione. Infine si è verificata la correttezza della funzione `borders` che descrive il comportamento dei Boid ai bordi dello spazio di simulazione. Per quanto riguarda il testing della classe Flock, come sempre si sono testate le varie funzioni definite in modo tale da poter comprendere quanti più casi possibili e cercare di trovare errori nelle implementazioni attuate. I test in questo caso hanno permesso di individuare un errore nell'implementazione delle funzioni `updatePosition` e `updateVelocity` in quanto le posizioni e le velocità dei Boid all'interno del flock non venivano aggiornate correttamente senza la creazione del vettore aggiuntivo nel quale vengono contenuti i nuovi valori. Le funzioni del calcolo delle medie e dell'incertezza sono state testate facendo attenzione che non assumessero valori indefiniti nei casi particolari di nessuno o un singolo Boid, e poi si è passato al testare la funzione `countBoidsInFlock`. Essendo una funzione importante nel nostro progetto ed essendo particolarmente complessa, ci siamo voluti accertare del suo corretto funzionamento svolgendo vari test per molte situazioni. E' grazie ad essi che abbiamo notato alcuni errori nell'implementazione della funzione e siamo riusciti a correggerli, in particolare perché abbiamo notato che per via di un errore di ordine nella logica del codice essa non restituiva il numero di Boid nei vari flock che ci aspettavamo. Una volta individuato l'errore infatti, siamo riusciti a correggerlo e ci siamo accertati che ora la nuova funzione passasse tutti i test. Infine la funzione `simulate` non è stata testata direttamente con doctest, ma essendo una funzione che deve stampare a schermo i vari risultati ottenuti, si è semplicemente controllato che essa stampasse i valori attesi dalle varie funzioni eseguendo il programma dal terminale, modificando volta per volta il `main.cpp` e facendo un grande numero di prove.

Compilazione e utilizzo di CMake:

Per la compilazione del nostro programma abbiamo scelto di utilizzare CMake. Esso è un comodo strumento utilizzabile su diverse piattaforme che permette di automatizzare il processo di generazione e compilazione di progetti software, e che abbiamo sfruttato anche per la formattazione dei vari file e per la verifica dei test unitari. CMake è nato per i progetti scritti in codice C e C++ ma ora è anche utilizzabile per altri linguaggi di programmazione come Python, e utilizza una sintassi dichiarativa che si basa sul file di configurazione CMakeLists.txt, nel quale vanno specificate le varie impostazioni di compilazione desiderate per il proprio progetto. Nel nostro caso abbiamo richiesto l'utilizzo di un linguaggio C++17 standard, ed abbiamo abilitato la generazione di messaggi diagnostici con le opzioni di compilazione -Wall -Wextra. Inoltre si è aggiunta l'opzione di "AddressSanitizer" e "UndefinedBehaviourSanitizer" nella modalità di debug perché grazie ad essi è possibile individuare e risolvere errori come accessi fuori limite alla memoria e comportamenti indefiniti (è proprio per uno di questi errori di buffer-overflow e di allocazione in memoria non valida che si è scelto di modificare l'implementazione della funzione countBoidsInFlock e di utilizzare uno stack). Per prima cosa quindi abbiamo configurato il file CMakeLists.txt e successivamente lo abbiamo utilizzato per creare l'area di compilazione apposita chiamata "build" utilizzando il comando:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
```

In particolare, questo comando genera i file di build necessari per il nostro progetto impostando la variabile CMAKE_BUILD_TYPE in modalità di debug, il che ci ha consentito di avere informazioni più dettagliate sugli eventuali errori. Va specificato che al termine della scrittura del codice, per configurare una directory build che fosse in grado di compilare più velocemente e produrre eseguibili di prestazioni migliori, abbiamo modificato tale comando in modo tale da impostare la variabile CMAKE_BUILD_TYPE in modalità di release. Fatto questo si è dunque compilato il progetto con il comando "cmake --build build". Questo comando esegue la compilazione dell'area all'interno della cartella "build" creata precedentemente. Questa directory aggiuntiva è comoda perché permette di separare i file contenenti il codice dagli eseguibili generati attraverso il processo di compilazione. Dopo la compilazione viene infatti generato l'eseguibile main nella directory build configurata appositamente da CMake. Grazie a CMake anche l'esecuzione dei test è risultata semplice, in quanto è sufficiente aggiungere al file di configurazione il supporto di testing scrivendo "include(CTest)" e successivamente aggiungere le istruzioni per la creazione dell'eseguibile e per l'aggiunta di tale eseguibile alla lista dei test. Oltre che il file in cui sono presenti i test, per costruire l'eseguibile è necessario aggiungere anche i file sorgente contenenti il codice vero e proprio utilizzato nei test. Questo è stato fatto per ognuno dei tre file di testing scritti. I test sono dunque stati eseguiti volta per volta con il comando "cmake --build build --target test". Infine si è passati all'automatizzazione della formattazione. CMake infatti consente di aggiungere un target personalizzato a partire da un file eseguibile presente nella directory di lavoro, e per questo è risultato comodo come strumento di formattazione il file .clang-format, scaricato con impostazioni predefinite. Una volta aggiunta l'istruzione al file di configurazione CMakeLists.txt ed eseguito il comando "cmake --build build --target format", CMake ci ha quindi permesso di formattare tutti i file sorgente in maniera automatica.

Gestione di input e output:

La gestione di input e output è stata interamente svolta nel file main.cpp, in quanto esso è l'unico con cui l'utente che esegue il programma dovrebbe poter interagire. All'interno della funzione principale main per prima cosa sono state dichiarate le variabili i cui valori vengono presi in input, e successivamente sono stati definiti i valori massimi di alcune variabili per evitare che l'utente possa inserire valori troppo elevati, che possono potenzialmente causare un rallentamento eccessivo del programma o il suo arresto. Il numero di Boid e la dimensione dello spazio infatti sono stati limitati per evitare che le funzioni raggiungessero tempi di computazione troppo lunghi, mentre il tempo di simulazione per evitare che il programma stampi sul terminale una quantità eccessiva di valori. Dopodiché abbiamo sfruttato la libreria "iostream" della libreria standard (inclusa nel boid.hpp), che si basa sul concetto di flussi di input e di output. Infatti attraverso la

sintassi `std::cout`, unita con l'operatore `<<`, la funzione stampa a schermo le varie istruzioni del programma, mentre scrivendo `std::cin` e `>>` la funzione prende in input il valore di una certa variabile precedentemente dichiarata. Per stampare a schermo in maniera controllata le istruzioni della fase di input del programma si sono incluse le librerie aggiuntive "thread" e "chrono", che abbiamo sfruttato affinché le stringhe venissero stampate sul terminale dopo qualche secondo in modo da lasciare all'utente il tempo di leggere le istruzioni. Dunque una volta giunti alla fase di input, abbiamo deciso di limitare i valori immessi con dei cicli while, i quali controllano se l'input è accettabile e, finché non lo è, ripuliscono ciò che è stato inserito e restituiscono un messaggio di errore. Per il numero di Boid è richiesto un numero intero, per cui ogni valore decimale viene automaticamente convertito in intero, ignorando i numeri dopo la virgola, e si ignorano anche tutti i caratteri non numerici come lettere o simboli scritti dopo il valore numerico. Abbiamo utilizzato questa sintassi del ciclo while per ogni valore preso in input, cambiando però di caso in caso i requisiti richiesti. Per esempio per il tempo di simulazione deve essere fornito un valore almeno maggiore del più piccolo intervallo di tempo, utilizzato per calcolare gli aggiornamenti di posizione e di velocità, che abbiamo impostato a 0,1s. Questo perché il programma deve dare modo di poter rilevare istante dopo istante i passaggi dei Boid da un flock a un altro, ma anche per non avere velocità troppo alte che poi andrebbero a fissarsi al valore massimo di 10. Come limite delle dimensioni dello spazio di simulazione abbiamo scelto di avere come superficie massima un quadrato 100x100 il quale consente, anche se si inserisce il numero massimo di Boid, che essi non siano troppo vicini e sovrapposti gli uni sugli altri, ma neanche troppo lontani per non far attivare le regole di volo, a meno che si imposti un raggio di percezione molto basso. Una volta presa in input la dimensione dello spazio, i Boid vengono generati in tale spazio con una posizione casuale con un semplicissimo ciclo for. Successivamente vengono presi in input le due distanze di percezione e di separazione e i parametri delle regole di volo. Eccetto per il fatto che la distanza di percezione deve essere minore della dimensione dello spazio e che la distanza di separazione deve essere minore di quella di percezione, non si sono imposti tanti limiti su questi parametri. Questo è voluto perché si vuole consentire all'utente di modificare i vari parametri e provare a prevedere quale sarà il loro effetto sul comportamento dello stormo. Per esempio riducendo il raggio di percezione si renderanno i Boid più "ciechi" il che renderà più difficile la formazione di uno stormo, oppure se si aumenta il parametro di coesione si formeranno stormi più coesi ovvero con una distanza media minore. Questi sono solo alcuni esempi di come si potrebbe comportare lo stormo a seconda di input particolari. Detto questo, una volta presi in input i valori dei parametri, la funzione main chiama la funzione `updateBoidParameters` definita nel `flock.cpp` la quale aggiorna le caratteristiche di ogni Boid con un ciclo for-each. Infine viene chiamata la funzione `simulate` che utilizza il valore preso in input del tempo per determinare quante iterazioni fare (`simulationSteps` corrisponde al tempo diviso la cadenza di tempo di ogni aggiornamento preimpostata a 0,1s). La funzione `simulate` dunque per ogni iterazione stampa a schermo il tempo, la distanza media tra i Boid e la loro velocità media con rispettive incertezze, il numero di flock rilevati, e il numero di Boid presenti in ciascuno di questi flock. Una volta arrivata al tempo impostato la funzione `simulate` si ferma e la funzione main termina con successo (il che viene descritto con la sintassi "return 0" la quale indica che il programma è terminato correttamente).

Esecuzione del programma e risultati attesi:

Per eseguire il programma in una propria directory, oltre ai file sorgente e header delle 3 classi con i rispettivi file di test, devono essere presenti il file `main.cpp`, il file `CMakeLists.txt` precedentemente configurato, il file `doctest.h` per l'esecuzione dei test e il file `.clang-format` per la formattazione. Quando si hanno questi file va per prima cosa configurata l'area di compilazione nella directory build, il che viene fatto con il comando:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
```

Ora che abbiamo configurato correttamente i file di build tramite CMake, si possono eseguire i comandi di formattazione e compilazione per produrre con successo l'eseguibile principale "main" nella directory build. I comandi in questo caso sono rispettivamente:

```
cmake --build build --target format
```

```
cmake --build build
```

A questo punto è già possibile eseguire il programma con il comando:

```
./build/main
```

Esso esegue il file binario "main" presente nella directory build e fa partire il programma. Se si vuole, prima di far partire il programma è anche possibile eseguire i test e controllare che passino con successo con il comando:

```
cmake --build build --target test
```

Detto questo, quando si esegue il programma appariranno a schermo le varie indicazioni per inserire in input i valori e i parametri della simulazione desiderati. Come prima cosa vengono chiesti il numero di Boid, il tempo di simulazione e la dimensione dello spazio della simulazione, dopodiché è necessario inserire le caratteristiche dei singoli Boid, ovvero il loro raggio di percezione, la distanza di separazione e i fattori delle 3 regole di volo. Se si introducono valori errati il programma spiega meglio quali sono i requisiti del valore richiesti e consente di riprovare. Una volta terminata la presa in input il programma stamperà a schermo tutta una serie di dati relativi alla simulazione, ovvero il tempo, la distanza tra i Boid media con la sua incertezza, la velocità dei Boid media con la propria incertezza, il numero di flock e il numero di Boid in ciascun flock. Consigliamo una volta terminato il programma di riprendere i dati dall'inizio e osservare l'evoluzione del numero di flock e del numero di Boid, così come la progressiva diminuzione della distanza e la stabilizzazione del valore di velocità media (nel caso in cui i valori immessi consentano un normale funzionamento del programma ovviamente).

-Un esempio indicativo di parametri standard che consentono di avere una simulazione corretta di uno stormo è il seguente (nel programma al posto delle virgole va inserito il punto per avere numeri decimali):

Numero di Boid: 50; Tempo di Simulazione: 30; Spazio di Simulazione: 50; Raggio di percezione: 8; Distanza di Separazione: 1; Fattore di Separazione: 1,5; Fattore di Coesione: 2,4; Fattore di Allineamento: 1.

Questi valori consentono di vedere l'evoluzione della simulazione da uno stadio iniziale in cui i Boid essendo situati in posizioni casuali sono piuttosto separati e distanti, a uno stato intermedio in cui essi muovendosi con velocità maggiori iniziano a raggrupparsi in diversi stormi, per poi arrivare a uno stadio finale in cui gli stormi minori si sono aggregati per formare stormi più grandi e si dovrebbe terminare con una situazione in cui sono rimasti solo due o tre stormi. In alcuni casi si può notare che l'incertezza sulla distanza media è molto grande, addirittura maggiore del valore medio. Questo è possibile e indica il fatto che le nostre unità statistiche (le distanze) sono molto diverse tra loro.

-Un'altra serie di valori standard che permette di avere situazioni analoghe ma con più Boid da gestire in uno spazio più esteso è la seguente:

Numero di Boid: 100; Tempo di Simulazione: 30; Spazio di Simulazione: 80; Raggio di percezione: 12; Distanza di Separazione: 2; Fattore di Separazione: 1; Fattore di Coesione: 1,5; Fattore di Allineamento: 0,75.

-Un caso particolare può essere invece quello di avere uno stormo molto coeso, il che comporta che i Boid saranno molto uniti e si dovrebbe osservare un valore di distanza media piuttosto basso. Un esempio di valori per raggiungere questa situazione può essere il seguente:

Numero di Boid: 20; Tempo di Simulazione: 10; Spazio di Simulazione: 20; Raggio di percezione: 5; Distanza di Separazione: 1; Fattore di Separazione: 1; Fattore di Coesione: 3; Fattore di Allineamento: 2.

-Per ottenere invece una situazione opposta, in cui i Boid rimangono piuttosto separati e risultano valori di distanza più alti, è possibile aumentare il fattore di separazione in questo modo:

Numero di Boid: 80; Tempo di Simulazione: 20; Spazio di Simulazione: 100; Raggio di percezione: 15; Distanza di Separazione: 3; Fattore di Separazione: 3; Fattore di Coesione: 1,2; Fattore di Allineamento: 0,8.

-Un ultimo caso che vogliamo presentare è quello in cui i Boid sono “ciechi”, e non sono dunque in grado di raggrupparsi in degli stormi numerosi e stabili. Per raggiungere questa situazione basta impostare un raggio di percezione molto basso rispetto allo spazio in si muovono i Boid. Quello che ci si aspetta quindi è che il programma restituisca un grande numero di stormi diversi, i quali saranno tutti formati da un numero limitato e continuamente variabile di Boid. Eccone i valori:

Numero di Boid: 20; Tempo di Simulazione: 10; Spazio di Simulazione: 30; Raggio di percezione: 2; Distanza di Separazione: 1; Fattore di Separazione: 1,5; Fattore di Coesione: 1,5; Fattore di Allineamento: 1.